

Towards a General Interface for SZ-DBC

Implementation Plan for Dimension-Agnostic, Automated
Detailed Balance Verification of Markov Chain Algorithms

S. Whitby

June 2026

Contents

1	Introduction and Motivation	3
2	Architecture	3
3	Algorithm File Interface	4
3.1	Mandatory declarations	4
3.2	Optional declarations	4
3.3	What must NOT appear	5
3.4	Minimal template	5
4	Automated Seed-State Sweep	6
4.1	New command-line flags	6
4.2	Sweep structure	6
4.3	Canonical type multisets	6
4.4	Canonical seed state	6
4.5	Output format	7
5	D-Dimensional Generalisation of the Core Library	7
5.1	State format	7
5.2	State normalisation	7
5.3	τ -BFS in D spatial dimensions	7
5.4	Translation group in D dimensions	8
5.5	Rotational symmetry	8
5.6	State enumeration	9
6	Phase-Space Algorithms	9
6.1	Why phase-space matters	9
6.2	State format for phase-space algorithms	9
6.3	The streaming-collision problem	10
6.4	Archetypal phase-space algorithm: position-velocity Metropolis	10
7	Foreseeable Issues	14
7.1	Lazy evaluation of grid-dependent globals	14

7.2	EBE cost across many system sizes	14
7.3	Phase-space state space growth	14
7.4	τ -BFS with four-dimensional coordinates	14
7.5	Hard-core constraint for phase-space systems	15
7.6	Coupling atoms with kinetic energy	15
8	Effort and Difficulty Assessment	16
9	Summary	16

1 Introduction and Motivation

The current SZ-DBC checker verifies detailed balance for a single, fixed system configuration: one specific lattice size, one specific particle multiset, one specific spatial dimension (2D). These parameters are all hard-coded in the algorithm file itself. This design conflates two concerns that should be separate: the *physics* of the algorithm under test, and the *infrastructure* needed to run the test.

This report describes a comprehensive plan to give SZ-DBC a general, dimension-agnostic interface. The goal is threefold:

1. An algorithm file should contain only the essential physics: the spatial dimension, the particle types, the transition rule, the energy function, and the coupling atoms. Everything else — seed state construction, lattice size, particle count — is generated automatically by the checker.
2. The checker should sweep over all relevant system sizes and particle counts up to user-specified maxima, running the full BFS and DB verification at each point and aggregating the results into a single PASS/FAIL verdict.
3. The framework should extend naturally to higher-dimensional systems, including *phase-space* algorithms where each particle carries both a position and a momentum (or velocity), enabling verification of a richer class of physically important algorithms such as lattice kinetic Monte Carlo methods.

Section 2 describes the three-layer architecture. Section 3 specifies the new algorithm file interface. Section 4 describes the automated seed-state sweep. Section 5 covers the D -dimensional generalisation of the core library. Section 6 addresses phase-space algorithms with position and momentum, including the design of an archetypal example algorithm. Section 7 catalogues foreseeable implementation issues. Section 8 gives a difficulty and effort assessment.

2 Architecture

The reformed codebase has three clearly separated layers, each with a well-defined responsibility.

Layer	File	Owns
Algorithm	<code>my_algo.wl</code>	physics: energy, transition rule, coupling atoms, spatial dimension
Orchestrator	<code>check.wls</code>	sweep strategy: which sizes and particle counts to test
Core library	<code>dbc_core.wl</code>	generic machinery: BFS, orbit computation, DB check

Nothing from the middle row should appear in the top row, and nothing algorithm-specific should be in the bottom two rows. This is precisely what the current design violates: every algorithm file sets `$nGrid`, `$particleTypes`, and `$seedState`, and the core library hard-codes 2D position vectors throughout.

The two documents that define the interface boundary are:

- `algorithm_requirements.md` — the contract every algorithm file must satisfy.
- `template.wl` — a minimal working algorithm file that a user copies and fills in.

3 Algorithm File Interface

3.1 Mandatory declarations

`$AlgDim = D`. The spatial dimension as a **hard-coded integer** — the only non-symbolic, non-overridable value in the algorithm file. It cannot be made symbolic because it determines the shape of all seed states, the structure of the translation group, and the form of the τ -BFS offset. For a pure 2D system `$AlgDim = 2`; for a 2D position + 2D momentum system, still `$AlgDim = 2` (the spatial dimension).

`$particleTypeSet = { $t_1, t_2, \dots$ }`. The set of distinct particle types the algorithm recognises. The checker will generate all canonical type multisets from this set for each particle count tested.

`Algorithm[state_]`. The MCMC transition rule. Must use `$nGrid` as a *free variable* — a symbol whose value is set by the checker before each run. Must not hard-code any specific lattice size internally.

`energy[state_]`. The bare energy (no β factor), with the same free-variable convention for `$nGrid`.

`DynamicSymParams[states_]`. Returns the association of symbolic coupling atoms:

```
<| "couplings" -> {couplingJ[1,2,1], couplingJ[1,2,2], ...},
  "extraSymParams" -> {} |>
```

Called once per run with the current state set.

3.2 Optional declarations

`$symmetryGroup`. Defaults to `{"translation"}`. The checker verifies any declared symmetry before exploiting it.

`$periodicDims`. Which coordinate components have periodic boundary conditions. Defaults to `Range[$AlgDim]` (all spatial components). For a phase-space system with 4D coordinates $\{x, y, v_x, v_y\}$, set `$periodicDims = {1, 2}` so that only the spatial components receive the τ -offset and modular normalisation. This is the mechanism by which the framework generalises to phase-space algorithms without structural changes to the BFS engine.

`$velocitySet`. For phase-space algorithms: the finite set of allowed velocity vectors, e.g. $\{-1, 0\}, \{1, 0\}, \{0, -1\}, \{0, 1\}$. The checker uses this to enumerate the full phase-space site set $\{1, \dots, n\}^{\$AlgDim} \times \$velocitySet$ when generating states.

numBeta. Inverse temperature for numerical mode only.

3.3 What must NOT appear

The algorithm file must *not* set \$nGrid to a specific value, define \$seedState, or define \$particleTypes. These are generated by the checker.

Backward compatibility. If the checker detects that the algorithm file sets \$nGrid, it runs in *single-size mode*: the declared \$nGrid, \$particleTypes, and \$seedState are used as-is, and the sweep is skipped. All existing algorithm files in the `examples/` directory work unchanged.

3.4 Minimal template

```
(* ==== template.wl -- minimal algorithm file ====
   Copy this file and fill in the three required sections. *)

$AlgDim          = 2;                                (* ONLY hard-coded value
*)
$particleTypeSet = {1, 2};                            (* distinct particle types
*)
$symmetryGroup   = {"translation", "D4"};

(* ---- Energy ---- *)
energy[state_] := Module[{...},
  (* Use $nGrid as a free variable for lattice size.
     All spatial operations must use pairwise differences. *)
  ...]

(* ---- Algorithm ---- *)
Algorithm[state_List] := Module[{n = $nGrid, ...},
  (* One MCMC step. Call RandomReal[], RandomChoice[], etc. as
     needed.
     Do NOT hard-code n = 3 or any specific lattice size. *)
  ...]

(* ---- Symbolic parameters ---- *)
DynamicSymParams[states_List] :=
  Module[{types, couplings},
    types = Sort @ DeleteDuplicates @ Flatten[#[[2]] & /@ # & /@
      states, 1];
    couplings = Flatten @ Table[
      couplingJ[a, b], {a, types}, {b, a, Max[types]};
    <| "couplings" -> couplings, "extraSymParams" -> {} |>]
```

4 Automated Seed-State Sweep

4.1 New command-line flags

Flag	Meaning
<code>-maxN N</code>	Maximum lattice side length (default 3)
<code>-maxParticles K</code>	Maximum particles per seed state (default 4)
<code>-allTypes</code>	Test all canonical type multisets (default: one per count)

4.2 Sweep structure

For each lattice side length $n = 1, 2, \dots, \text{maxN}$:

1. Compute the number of available sites: $N_{\text{sites}} = n^{\text{\$AlgDim}} \times |\text{\$velocitySet}|$ (the $|\text{\$velocitySet}|$ factor is 1 for pure-position systems).
2. For each particle count $k = 0, 1, \dots, \min(\text{maxParticles}, N_{\text{sites}})$:
 - (a) Generate one or more canonical type multisets of size k from $\text{\$particleTypeSet}$ (see below).
 - (b) For each such multiset T : set $\text{\$nGrid} = n$, $\text{\$particleTypes} = T$, generate the canonical seed state, and run the full Steps 1–8 pipeline.
 - (c) Record PASS/FAIL for the triple (n, k, T) .

After all runs: print an aggregated table; exit with code 0 if all passed, code 1 otherwise.

4.3 Canonical type multisets

Given particle count k and type set $\{t_1, t_2, \dots\}$, the default canonical multiset cycles through types in order: $T = (t_1, t_2, t_1, t_2, \dots)$ truncated to length k . For two types $\{1, 2\}$ and $k = 5$: $T = (1, 2, 1, 2, 1)$.

Why this default matters. With k particles cycling through all available types, every pair of distinct types appears in the system for even k . This ensures that all coupling atoms $\text{couplingJ}[a, b, d^2]$ can appear in the transition probabilities, making the DB check sensitive to all coupling parameters. The `-allTypes` flag generates every distinct multiset for thorough checking.

4.4 Canonical seed state

Given lattice side length n , dimension D , and type multiset $T = (t_1, \dots, t_k)$, the canonical seed state places particle i at the i -th position in lexicographic order on $\{1, \dots, n\}^D$:

$$\text{seedState} = \{ \{(1, 1, \dots, 1), t_1\}, \{(1, 1, \dots, 2), t_2\}, \dots \}$$

For a 2D lattice of side 3 with $T = (1, 2)$: $\text{seed} = \{((1, 1), 1), ((1, 2), 2)\}$. This is deterministic, dimension-agnostic, and independent of any physics.

For phase-space systems, the seed also specifies initial velocities: each particle is assigned the first element of $\text{\$velocitySet}$ (e.g., $(-1, -1)$), giving the lexicographically first phase-space state.

4.5 Output format

The checker prints a table after completing all runs:

n=1, k=0, T={}	: tau	PASS	D4	--	Erg	--	DB	PASS
n=1, k=1, T={1}	: tau	PASS	D4	--	Erg	PASS	DB	PASS
n=2, k=0, T={}	: tau	PASS	D4	PASS	Erg	--	DB	PASS
n=2, k=1, T={1}	: tau	PASS	D4	PASS	Erg	PASS	DB	PASS
n=2, k=2, T={1,2}	: tau	PASS	D4	PASS	Erg	PASS	DB	PASS
n=3, k=2, T={1,2}	: tau	PASS	D4	PASS	Erg	PASS	DB	PASS
n=3, k=3, T={1,2,1}	: tau	PASS	D4	PASS	Erg	PASS	DB	PASS

OVERALL: PASS (7/7 configurations)								

5 D -Dimensional Generalisation of the Core Library

5.1 State format

The current format $\{\{\text{row}, \text{col}\}, \text{type}\}$ is replaced by $\{c_1, c_2, \dots, c_{D_{\text{total}}}\}, \text{type}\}$ where $D_{\text{total}} = \$\text{AlgDim} + \MomDim . For a pure 2D system $D_{\text{total}} = 2$; for a 2D spatial + 2D velocity system $D_{\text{total}} = 4$.

This change is backwards-compatible: any function that currently indexes `state[[i,1,1]]` (row) and `state[[i,1,2]]` (column) continues to work for $D = 2$. The internal functions must be generalised to loop over the coordinate vector.

5.2 State normalisation

```
(* Current (hardwired 2D) *)
$dbcNormState[state_, nGrid_] :=
  SortBy[Map[{Mod[#[[1,1]]-1,nGrid]+1,
               Mod[#[[1,2]]-1,nGrid]+1}, #[[2]]} &, state], First
        ]

(* Generalised: periodicDims specifies which components wrap *)
$dbcNormState[state_, nGrid_, periodicDims_] :=
  SortBy[Map[Function[p,
    {MapIndexed[
      If[MemberQ[periodicDims,#2[[1]]], Mod[#1-1,nGrid]+1, #1]
      &,
      p[[1]]],
      p[[2]]}], state], First]
```

The `periodicDims` parameter controls which coordinate components are wrapped by the modular arithmetic. For pure position systems it equals `Range[AlgDim]`; for phase-space systems it is the subset $\{1, \dots, \text{AlgDim}\}$.

5.3 τ -BFS in D spatial dimensions

The translational invariance check adds symbolic offsets $\tau_1, \dots, \tau_D$ to the spatial components of every position vector. Generically, for spatial dimension D , the checker introduces

D symbols $\{\tau_1, \dots, \tau_D\}$ and augments only the periodic components:

```
(* Generalised tau-augmentation *)
$dbcAddTau[state_, alg_dim_] :=
  Map[Function[p,
    {MapIndexed[
      If[#2[[1]] <= alg_dim, #1 + Symbol["tau"<>ToString
        [#2[[1]]]], #1] &,
      p[[1]]],
    p[[2]]}], state]
```

For a 2D system this recovers the current $\{\tau_r, \tau_c\}$ behaviour. For a phase-space system with coordinates $\{x, y, v_x, v_y\}$ and $\$AlgDim = 2$, the augmented positions are $\{x + \tau_1, y + \tau_2, v_x, v_y\}$ — velocity components are not shifted, correctly capturing that translational invariance is a property of position space alone.

5.4 Translation group in D dimensions

The group of lattice translations $\mathbb{Z}_n^D$ is generated by all shifts $\mathbf{d} = (d_1, \dots, d_D)$ with $d_i \in \{0, 1, \dots, n-1\}$. Currently the group elements are enumerated by a 2D Table over $\{\mathbf{dr}, \mathbf{dc}\}$; the generalisation is a D -fold Cartesian product:

```
(* Generalised translation elements *)
translationElems[nGrid_, alg_dim_] :=
  Map[<| "shift" -> # |> &,
    Flatten[Outer[List, Sequence @@ Table[Range[0, nGrid-1],
      alg_dim]], alg_dim-1]]
```

The shift vectors act on the spatial components of position only; velocity components are unchanged.

5.5 Rotational symmetry

For $D = 2$: the D4 group (8 elements) is already implemented. No change.

For $D = 3$: the octahedral group O_h has 48 elements. These can be enumerated as the set of all 3×3 signed permutation matrices with determinant +1. Adding Oh to the checker is a well-defined but non-trivial extension; it is deferred to a later phase.

For phase-space systems ($D = 2$ spatial, $D = 2$ velocity): a D4 rotation by 90° clockwise maps $(x, y) \rightarrow (y, n+1-x)$ in position space and simultaneously maps $(v_x, v_y) \rightarrow (v_y, -v_x)$ in velocity space. This is a joint transformation on the full coordinate vector. The symmetry element must encode both actions:

```
(* D4 rotation for a 2D-position + 2D-velocity state *)
$dbcPhaseSpaceD4Pos[1, x_, y_, vx_, vy_, n_] :=
  {y, n+1-x, vy, -vx} (* 90 degrees CW *)
```

For the initial implementation of phase-space systems, translation-only orbits are sufficient. The coupled D4 action can be added in a later phase.

5.6 State enumeration

```
(* Current: 2D grid positions only *)
$dbcEnumerateNParticleStates[seedState_, nGrid_] :=
  Module[{types, allPos},
    types = Sort[#[[2]] & /@ seedState];
    allPos = Flatten[Table[{r,c},{r,nGrid},{c,nGrid}],1];
    ...]

(* Generalised: D-dimensional grid, optional velocity set *)
$dbcEnumerateNParticleStates[types_, nGrid_, alg_dim_,
  velocity_set_:{}] :=
  Module[{posSites, sites, allAssignments},
    posSites = Flatten[
      Outer[List, Sequence @@ Table[Range[nGrid], alg_dim]],
      alg_dim - 1];
    sites = If[velocity_set === {},
      posSites,
      Flatten[Outer[Join, posSites, velocity_set, 1], 1]];
    (* Enumerate all N-particle assignments on 'sites' *)
    ...]
```

For a 2D lattice of side n with 4-direction velocities, this produces $4n^2$ sites. For $n = 2$ and $N = 2$ particles: $C(16, 2) = 120$ two-particle states (assuming particles are distinguishable by type).

6 Phase-Space Algorithms

6.1 Why phase-space matters

Most existing MCMC algorithms in statistical physics evolve only the positions of particles; velocities or momenta are integrated out analytically. A richer class of algorithms, including stochastic lattice gases and kinetic Monte Carlo methods, maintains both positions and velocities as dynamical variables and samples from the full phase-space Boltzmann distribution

$$\pi(\mathbf{r}, \mathbf{v}) \propto \exp(-\beta H(\mathbf{r}, \mathbf{v})), \quad H = \sum_i \frac{1}{2} |\mathbf{v}_i|^2 + \sum_{i < j} V(|\mathbf{r}_i - \mathbf{r}_j|^2).$$

Verifying detailed balance for these algorithms requires the checker to handle states with a velocity component, which the current design does not support.

6.2 State format for phase-space algorithms

Each particle is described by a four-component coordinate vector $\{x, y, v_x, v_y\}$ plus a type. The full state is a sorted list of such pairs:

```
{ {{1, 1, -1, 0}, 1},
  {{2, 3, 0, 1}, 2},
  ... }
```

Sorting is by the full coordinate vector lexicographically: positions are primary, velocities are secondary. This generalises the current sort without any conceptual change.

The algorithm file declares:

```
$AlgDim      = 2;                (* x, y are the two spatial dims
*)
$periodicDims = {1, 2};          (* only x, y are periodic *)
$velocitySet  = {{-1,0},{1,0},{0,-1},{0,1}}; (* allowed
velocities *)
```

6.3 The streaming–collision problem

The most physically natural algorithm for phase-space systems is streaming–collision dynamics:

1. **Streaming:** each particle moves $\mathbf{r}_i \rightarrow \mathbf{r}_i + \mathbf{v}_i \pmod{n}$.
2. **Collision:** at sites with multiple particles, redistribute velocities stochastically.

However, constructing a streaming–collision algorithm that satisfies detailed balance with the *full* Hamiltonian $H = K + V$ is non-trivial. The streaming step is deterministic and maps the system from one configuration to another with different potential energy $V(\mathbf{r} + \mathbf{v}) \neq V(\mathbf{r})$. For the collision acceptance probability to restore detailed balance, it must account for *both* the kinetic energy change ΔK and the potential energy change ΔV induced by the preceding streaming. This couples the collision kernel to the global state in a way that makes the acceptance criterion complex and the DB verification non-trivial to design.

Specifically, for the combined step to satisfy

$$T(\mathbf{s} \rightarrow \mathbf{s}') \exp(-\beta H(\mathbf{s})) = T(\mathbf{s}' \rightarrow \mathbf{s}) \exp(-\beta H(\mathbf{s}')),$$

the collision acceptance ratio must equal $\exp(-\beta(H(\mathbf{s}') - H(\mathbf{s}))) = \exp(-\beta(\Delta K + \Delta V))$, where ΔV depends on the *post-streaming* particle positions. A collision kernel that uses only ΔK (the standard choice) satisfies detailed balance with respect to K alone — which is correct only when $V \equiv 0$ (non-interacting particles).

This is a genuine theoretical constraint, not a limitation of the checker. Any correct streaming–collision MCMC algorithm for interacting phase-space particles must either: (a) use a collision acceptance criterion that references the global potential energy, or (b) apply a separate global Metropolis correction step after streaming. Implementing such an algorithm cleanly is deferred to a future extension.

6.4 Archetypal phase-space algorithm: position–velocity Metropolis

The clean, pedagogically correct first example is a *position–velocity Metropolis* algorithm: two independent sub-moves, chosen at random, each of which satisfies detailed balance with respect to the full Hamiltonian $H = K + V$.

Algorithm description

At each step:

1. Choose a particle p uniformly at random.
2. Draw $u \sim \text{Uniform}[0, 1]$.
3. **If** $u < \frac{1}{2}$ (**position move**):
 - (a) Choose a displacement $\mathbf{d}$ uniformly from the set of nearest-neighbour directions.
 - (b) Proposed new position: $\mathbf{r}' = \mathbf{r}_p + \mathbf{d}$.
 - (c) *Hard-core rejection*: if any other particle already occupies $\mathbf{r}'$ (regardless of its velocity), return the current state.
 - (d) Compute $\Delta V = V_{\text{new}} - V_{\text{old}}$ (potential energy change; velocity of p is unchanged).
 - (e) Accept with $\min(1, e^{-\beta \Delta V})$; else stay.
4. **If** $u \geq \frac{1}{2}$ (**velocity move**):
 - (a) Choose a new velocity $\mathbf{v}'$ uniformly from `$velocitySet`.
 - (b) Compute $\Delta K = \frac{1}{2}(|\mathbf{v}'|^2 - |\mathbf{v}_p|^2)$.
 - (c) Accept with $\min(1, e^{-\beta \Delta K})$; else stay.

Why this satisfies detailed balance

The position move satisfies DB with respect to H :

$$T_{\text{pos}}(\mathbf{s} \rightarrow \mathbf{s}') e^{-\beta H(\mathbf{s})} = T_{\text{pos}}(\mathbf{s}' \rightarrow \mathbf{s}) e^{-\beta H(\mathbf{s}')}.$$

This holds because (i) the proposal is symmetric (same neighbour set forward and reverse), (ii) hard-core rejection is symmetric, and (iii) the Metropolis acceptance satisfies $\min(1, e^{-\beta \Delta V}) \cdot e^{-\beta K} = \min(1, e^{+\beta \Delta V}) \cdot e^{-\beta K} \cdot e^{-\beta \Delta V}$ exactly. Kinetic energy cancels because $\Delta K = 0$ for a position move.

The velocity move satisfies DB with respect to H :

$$T_{\text{vel}}(\mathbf{s} \rightarrow \mathbf{s}') e^{-\beta H(\mathbf{s})} = T_{\text{vel}}(\mathbf{s}' \rightarrow \mathbf{s}) e^{-\beta H(\mathbf{s}')}.$$

This holds because the velocity proposal is symmetric over `$velocitySet`, potential energy cancels ($\Delta V = 0$ for a velocity move), and Metropolis on ΔK satisfies the condition exactly.

A 50/50 random mixture of two Markov kernels that both satisfy DB with respect to the same stationary distribution π is itself DB-satisfying with respect to π . Therefore the full algorithm satisfies detailed balance with respect to $\pi \propto \exp(-\beta H)$.

Algorithm file sketch

```
(* ==== phase_space_metropolis.wl ====
 2D spatial + 2D velocity Metropolis.
 State: {{{x, y, vx, vy}, type}, ...}
 H = sum_i (vx_i^2 + vy_i^2)/2 + sum_{i<j} couplingJ[ta,tb,|ri-
      rj|^2] *)
```

```

$AlgDim          = 2;
$periodicDims    = {1, 2};
$velocitySet     = {{-1,0},{1,0},{0,-1},{0,1}};
$particleTypeSet = {1, 2};
$maxD2           = 2;

$posDisps = {{-1,0},{1,0},{0,-1},{0,1}};

(* Potential energy: pairwise, depends on spatial positions only
*)
$potE[state_, n_] := Module[{total = 0},
  Do[
    With[{d2 = $pairD2Spatial[state[[i,1]], state[[j,1]], n]},
      If[0 < d2 <= $maxD2,
        total += couplingJ[state[[i,2]], state[[j,2]], d2]],
      {i, Length[state]}, {j, i+1, Length[state]}; total]

$pairD2Spatial[p1_, p2_, n_] :=
  With[{dr = p1[[1]]-p2[[1]], dc = p1[[2]]-p2[[2]]},
    With[{dra = Mod[Abs[dr],n], dca = Mod[Abs[dc],n]},
      Min[dra,n-dra]^2 + Min[dca,n-dca]^2]]

(* Kinetic energy: sum of |v|^2 / 2 per particle *)
$kinE[state_] := Total[(#[[1,3]]^2 + #[[1,4]]^2)/2 & /@ state]

energy[state_] := $kinE[state] + $potE[state, $nGrid]

Algorithm[state_List] :=
  Module[{n = $nGrid, p, x, y, vx, vy, tp, rest, u},
    If[state === {}, Return[state]];
    p = RandomChoice[state];
    x = p[[1,1]]; y = p[[1,2]];
    vx = p[[1,3]]; vy = p[[1,4]];
    tp = p[[2]];
    rest = DeleteCases[state, p, 1];
    u = RandomReal[];

    If[u < 1/2,
      (* --- Position move --- *)
      With[{d = RandomChoice[$posDisps],
        newX = {x,y} + RandomChoice[$posDisps]},
        (* hard-core: no other particle at newX regardless of
        velocity *)
        If[AnyTrue[rest, Function[q,
          Mod[q[[1,1]]-newX[[1]],n]===0 &&
          Mod[q[[1,2]]-newX[[2]],n]===0]],
          Return[state]];
        With[{ns = SortBy[Append[rest,
          {{newX[[1]],newX[[2]],vx,vy},tp]], First]},
          With[{dV = $potE[ns,n] - $potE[state,n]},

```

```

        If[RandomReal[] < Piecewise[{{1,dV<=0}},Exp[-\[Beta]*
            dV]],
            ns, state]]]],

(* --- Velocity move --- *)
With[{nv = RandomChoice[$velocitySet]},
    With[{dK = (nv[[1]]^2+nv[[2]]^2 - vx^2-vy^2)/2},
        With[{ns = SortBy[Append[rest,{{x,y,nv[[1]],nv[[2]]},tp
            }],First]},
            If[RandomReal[] < Piecewise[{{1,dK<=0}},Exp[-\[Beta]*
                dK]],
                ns, state]]]]]]

DynamicSymParams[states_List] :=
Module[{types, n, d2Vals, atoms},
    types = Sort @ DeleteDuplicates @ Flatten[#[[2]] & /@ # & /@
        states, 1];
    n = $nGrid;
    d2Vals = Sort @ DeleteDuplicates @ Select[
        Flatten @ Table[Min[dr,n-dr]^2+Min[dc,n-dc]^2,
            {dr,0,Floor[n/2]},{dc,0,Floor[n/2]}],
        0 < # <= $maxD2 &];
    atoms = Flatten @ Table[
        If[a<=b, Table[couplingJ[a,b,d2],{d2,d2Vals}], Nothing],
        {a,types},{b,types}];
    <| "couplings" -> atoms, "extraSymParams" -> {} |>]

```

What the checker will verify

When run with `-maxN 2 -maxParticles 2`, the checker will test:

- $n = 1, k = 0$: trivially empty state.
- $n = 1, k = 1$: one particle on a single site; only velocity moves are possible. The checker verifies DB for all velocity pairs $\{v, v'\}$ and all coupling values.
- $n = 2, k = 1$: one particle on a 2×2 torus. Position moves and velocity moves both active.
- $n = 2, k = 2, T = \{1, 2\}$: two particles on a 2×2 torus with 4 velocities each. State space has $C(16, 2) = 120$ states. The DB check exercises all coupling atoms $\text{couplingJ}[1, 2, d^2]$ and verifies that the kinetic acceptance is correctly applied.

A broken variant (e.g., using $\beta/2$ in the velocity acceptance criterion) will produce DB violations at the $k = 1$ level where only velocity moves operate, making the failure immediately legible.

7 Foreseeable Issues

7.1 Lazy evaluation of grid-dependent globals

Several existing algorithm files compute geometry helpers at load time using the then-current value of `$nGrid`:

```
$nStep      = Max[1, Round[physLen]];    (* vmmc_2d.wl *)
$smDisps    = DeleteCases[Flatten[...], {0,0}];    (*
    single_metroplis.wl *)
```

These are evaluated once when the file is `Get[]`'d and then cached as global constants. Under the sweep design, `$nGrid` changes between runs, so any helper that depends on `$nGrid` and is computed at load time will silently use a stale value. The fix is to compute such helpers lazily inside `Algorithm[]` or `energy[]` using the local `n = $nGrid` binding:

```
Algorithm[state_List] := Module[{n = $nGrid,
    disps = DeleteCases[Flatten[Table[{dx,dy},{dx,-1,1},{dy
    ,-1,1}],1],{0,0}],
    ...}, ...]
```

For the existing examples, inspection shows that the precomputed globals are all independent of `$nGrid` (e.g., `$smDisps` is the fixed list of 8 unit displacements). The sweep design is therefore backward compatible for these files without modification.

7.2 EBE cost across many system sizes

The EBE Phase 2 (hyperplane arrangement BFS via `FindInstance`) runs once per (n, T) combination. A sweep of `-maxN 3 -maxParticles 4` with two types produces roughly 20 independent check runs. Phase 2 is fast for small systems but the aggregate cost grows. The existing chamber-reuse mechanism (D4 EBE chambers reused for DB EBE) already eliminates duplicate Phase 2 work within a single (n, T) run. Cross-size reuse is not straightforward since the coupling atom set changes with particle types. The recommended mitigation is to fail fast: if any (n, T) run finds a DB violation, the sweep stops at that point.

7.3 Phase-space state space growth

Including velocity degrees of freedom multiplies the state space by $|\$velocitySet|$ per particle. For a 2×2 lattice with 4-direction velocities and 2 particles the state space is approximately 120, compared to 12 for the pure-position case. For $n = 3$ with 8 velocities and 3 particles the state space reaches several thousand. For phase-space systems the default `-maxN` should be 2, and the template file should carry a comment recommending this.

7.4 τ -BFS with four-dimensional coordinates

For a state with coordinate vector $\{x, y, v_x, v_y\}$, the `$dbcAddTau` function must add τ_1 to x and τ_2 to y only. If the `$periodicDims` parameter is not passed through correctly to all three τ -related functions (`$dbcAddTau`, `$dbcNormTauState`, and the norm function passed

to `$dbcBuildStateLeaves`), the τ symbols will either leak into the velocity components (causing a spurious τ -FAIL) or fail to cancel in the spatial components (also a spurious FAIL). Careful parameter threading is required.

7.5 Hard-core constraint for phase-space systems

There are two natural hard-core conventions: (a) no two particles at the same *position*, regardless of velocity, and (b) no two particles at the same *phase-space point* $\{x, y, v_x, v_y\}$. Convention (a) is physically appropriate for particles with a finite size; convention (b) models a fermionic exclusion principle. These lead to different state enumerations and different transition probabilities. The algorithm file must make this choice explicit, and the checker's state enumeration must respect it. For the canonical example (§6.4) convention (a) is used because the position move checks for occupancy based on position only.

7.6 Coupling atoms with kinetic energy

For a phase-space system the energy is $H = K + V$. The kinetic part $K = \sum_i |\mathbf{v}_i|^2/2$ uses the *velocity values as data* from the state, not as symbolic parameters. The coupling atoms returned by `DynamicSymParams` are therefore only the potential couplings $\{\text{couplingJ}[a, b, d^2]\}$; the factor $1/2$ in K is a fixed rational. This means `$dbcIsExpZero` will encounter terms with both rational exponents (from kinetic contributions) and symbolic exponents (from potential contributions). Fortunately `$dbcIsExpZero` already handles this: it groups terms by exponent class, and rational exponents form their own class. No changes to the DB check are needed.

8 Effort and Difficulty Assessment

Task	Difficulty	Notes
<code>algorithm_requirements.md</code> + <code>template.wl</code>	Low	Documentation only
Backward-compatible detection in <code>check.wls</code>	Low	Detect <code>\$nGrid</code> assignment
Multi-size sweep loop in <code>check.wls</code>	Medium	Nested loop; output table
Canonical type-multiset and seed-state generation	Medium	Deterministic, dimension-agnostic
D -dim state normalisation in <code>dbc_core.wl</code>	Medium	Pervasive; thread <code>periodicDims</code>
D -dim translation group elements	Low-Medium	D -fold Cartesian product
D -dim orbit computation	Medium	Only positional indexing changes
τ -BFS with <code>periodicDims</code>	Medium	Thread param into three functions
Phase-space state enumeration	Medium	Extend to velocity-set sites
<code>phase_space_metropolis.wl</code> example	Medium	Position and velocity moves clean
Coupled D4 on position+velocity	Hard	Defer to later phase
Streaming-collision MCMC	Hard	Non-trivial acceptance; defer
Adapt existing examples	Low	Remove 3 lines per file
Full regression testing	Medium	Verify sweep matches current output
Estimate (staged)		Core: 1–2 wks; phase-space example: 3–5 days

The recommended implementation order is:

1. Write `algorithm_requirements.md` and `template.wl` (establishes the interface contract before any code changes).
2. Generalise `dbc_core.wl` to D dimensions with `periodicDims` (enables all subsequent work).
3. Add the multi-size sweep to `check.wls` with backward-compatible mode.
4. Adapt existing examples and run regression tests.
5. Implement `phase_space_metropolis.wl` and test on $n = 2$.
6. Add coupled D4 action and streaming-collision example (later phases).

9 Summary

The principal architectural change is a clean separation of *physics* (algorithm file) from *infrastructure* (checker and core library). An algorithm file in the new design de-

clares only three physical items: the spatial dimension `$AlgDim`, the particle type set `$particleTypeSet`, and the three functions `Algorithm`, `energy`, and `DynamicSymParams`. Everything else — lattice size, seed state, particle count sweep — is generated and controlled by the checker.

The D -dimensional generalisation of the core library is largely mechanical: every function that currently indexes a 2-component position vector must be updated to loop over the `periodicDims` parameter. The BFS engine and the DB checking machinery require no changes.

Phase-space algorithms are accommodated naturally within this framework by treating velocity as additional (non-periodic) coordinate components. The archetypal example — a position+velocity Metropolis algorithm with separate sub-moves — is theoretically clean, straightforward to implement, and a valuable demonstration that the checker correctly handles a Hamiltonian with both kinetic and potential terms. The more complex streaming+collision case is deferred because constructing a DB-correct streaming+collision algorithm for *interacting* particles requires an acceptance criterion that accounts for potential energy changes due to streaming, making it a non-trivial algorithm design problem in its own right.